

Gonka Competitive Feature Matrix: Agent-Relevant Inference Provider Comparison

Version: 1.0 **Date:** 2026-04-01 **Audience:** Gonka leadership and GTM strategy team **Scope:** Comparison of Gonka vs OpenRouter vs OpenAI vs Anthropic vs Together AI across 8 agent-relevant dimensions

Executive Summary

OpenClaw developers building AI agents need inference providers that go beyond simple chat completions. Agents make 3-10x more LLM calls than chatbots, resend full context via heartbeats every 30 minutes, and increasingly require server-side state management, automatic model routing, and persistent memory. When evaluated against these agent-specific requirements, no single provider dominates across all dimensions.

Gonka wins on the two dimensions most unique to agent workloads – server-side session persistence and automatic model tiering – features that no other compared provider offers natively. However, Gonka loses decisively on model breadth (1 model vs OpenRouter’s 500+) and uptime/reliability (no SLA, unproven at scale). OpenAI leads on tool calling quality and infrastructure reliability. Anthropic leads on prompt caching economics. OpenRouter leads on model breadth. Together AI leads on raw pricing.

The overall score – Gonka 2/8, OpenRouter 1/8, OpenAI 3/8, Anthropic 2/8, Together AI 1/8 – reveals a fragmented competitive landscape where Gonka’s agent-native features represent a genuine differentiation opportunity, contingent on closing critical infrastructure and ecosystem gaps identified in the companion provider landscape document (gonka_provider_landscape_map.md).

Methodology

Each of the 8 dimensions below is evaluated from the perspective of an OpenClaw agent developer building production agent systems. Scoring follows a strict framework:

- **WIN:** The provider has a clear, measurable advantage for OpenClaw agent use cases on this dimension. The advantage must be structural (not just marketing), and must matter for agents specifically (not general LLM usage).
- **TIE:** The provider delivers comparable functionality. Minor differences exist but do not meaningfully affect agent development outcomes.
- **LOSE:** The provider has a clear disadvantage that creates friction, cost, or capability gaps for OpenClaw agent developers.

Scoring is framed from the agent developer’s perspective: “As an OpenClaw developer building a production agent, which provider serves me best on this dimension?” Each verdict includes one-line evidence. The framework deliberately avoids weighting dimensions – leadership should weight based on their strategic priorities.

All data points are sourced from provider documentation, pricing pages, and the research corpus in `.planning/research/`. Where data is uncertain, confidence levels are noted. Gonka’s pricing is flagged as TBD throughout, as final per-token rates have not been set.

Feature Matrix

Dimension	Gonka	OpenRouter	OpenAI	Anthropic	Together AI
Agent Sessions	WIN: Server-side session persistence via /v1/sessions API; state survives across requests without client re-sending context	LOSE: Stateless passthrough; all session management is client-side	TIE: Assistants API provides thread-based state, but it is a separate API surface from chat completions	LOSE: No session management; stateless chat completions only	LOSE: No session management; stateless API
Model Tiering / Auto-Routing	WIN: 3-tier auto-routing via X-Gonka-Tier header (lite/mid/full K2.5 quantization); agents select cost-performance tier per request	LOSE: Manual model selection from 500+ catalog; no automatic tier routing based on task complexity	LOSE: Manual model selection (GPT-4o, GPT-5.4, etc.); no auto-routing between tiers	LOSE: Manual model selection only; no auto-routing	LOSE: Manual model selection; no tiering mechanism
Tool Calling	TIE: K2.5 native tool calling with 200-300 sequential calls stable; OpenAI-compatible function calling format	TIE: Passes through each provider's tool calling implementation; quality varies by underlying model	WIN: Best-in-class function calling with structured outputs and guaranteed JSON schema compliance	TIE: Strong tool use with beta structured output features; reliable but less mature than OpenAI's	TIE: Supported for compatible models via OpenAI-compatible API; quality depends on model
Streaming	TIE: vLLM SSE streaming via OpenAI-compatible endpoint	TIE: SSE streaming passthrough from upstream providers	TIE: Native SSE streaming with full event types	TIE: Native SSE streaming	TIE: SSE streaming via OpenAI-compatible API
Memory / Context Management	TIE: /v1/memory API with TF-IDF search provides persistent key-value memory; functional but lower recall than vector-based approaches	LOSE: No memory management; stateless passthrough; all context management is client-side	WIN: Native prompt caching (automatic, 50% discount on cached input tokens); reduces effective cost of heartbeat context resends	WIN: Prompt caching with 90% discount on cached tokens (25% write premium); strongest caching economics for repeat contexts	LOSE: No memory or caching features; stateless API
Pricing Model	TIE: Per-token pricing (TBD – not yet published); potential cost advantage from decentralized compute but unverifiable until pricing is set	TIE: Per-token passthrough pricing + 5.5% credit markup; transparent but adds a hidden cost layer that increases effective per-token rates	TIE: Per-token pricing, transparent, with volume discounts available; premium pricing for proprietary models but predictable	LOSE: Per-token pricing at premium rates (\$15/\$75 per 1M tokens for Opus 4); highest cost per token among compared providers for agent workloads that make many calls	WIN: Per-token pricing on own GPU clusters with no middleman markup; verified cheapest for K2.5 at \$0.50/\$2.50 per 1M tokens
Uptime / Reliability	LOSE: No published SLA; unproven at production scale; single model on decentralized infrastructure with inherent variability from heterogeneous nodes	TIE: Established track record with multi-year operation; free tier throttled/degraded under load; no formal SLA but generally reliable	WIN: 99.9%+ SLA with global infrastructure; multi-region redundancy; the reliability benchmark that all other providers are compared against	WIN: Production SLA with reliable global infrastructure; consistent uptime track record; enterprise-grade operations	TIE: Own infrastructure with reliable track record; no formal SLA published but consistent availability on dedicated GPU clusters
Model Breadth	LOSE: 1 model (Kimi K2.5) in 3 quantization tiers; no alternative models for fallback or specialized tasks	WIN: 500+ models from all major providers; broadest selection available; single API key accesses every model	TIE: ~10 proprietary models (GPT-4o, GPT-5.4, GPT-4o mini, etc.); limited but high-quality selection covering most use cases	LOSE: ~5 models (Opus 4, Sonnet 4, Haiku 3.5); smallest catalog among compared providers	TIE: 200+ open models including K2.5, Llama 4, and fine-tuning options; strong open-model catalog

Summary Scores

Provider	Wins	Ties	Losses	Score
Gonka	2	4	2	2/8
OpenRouter	1	3	4	1/8
OpenAI	3	4	1	3/8
Anthropic	2	2	4	2/8
Together AI	1	4	3	1/8

Dimension Deep Dives

1. Agent Sessions

What OpenClaw agents need: When an OpenClaw agent processes a multi-step task – classifying a request, planning an approach, executing subtasks, synthesizing results – it makes multiple LLM calls that share conversational context. Without server-side session persistence, each call must re-send the full conversation history, inflating token costs. OpenClaw heartbeats compound this: every 30 minutes, the agent resends its full context to check for new tasks, costing approximately 9,600 tokens per heartbeat regardless of whether anything changed (source: OpenClaw pricing guides, .planning/research/STACK.md). Over 24 hours, heartbeats alone generate 48 full context re-sends.

How each provider delivers:

Gonka offers a dedicated `/v1/sessions` API (built in v1.2) where agents create a session via `X-Gonka-Session-ID` header, and the server maintains conversation state across requests. Subsequent calls within the same session do not need to re-send prior messages – the server appends them automatically. This is a genuine architectural advantage for agent workloads, directly reducing the token overhead of heartbeats and multi-step tasks. However, the current implementation stores sessions in-memory (v1.2 tech debt), meaning sessions are lost on server restart – a production reliability concern that partially undermines the feature’s value proposition.

OpenRouter is a stateless passthrough proxy. It receives a request, routes it to the upstream provider, and returns the response. No conversation state is maintained between requests. Agents using OpenRouter must manage their own session state client-side, re-sending full conversation history on every call. For OpenClaw agents with 30-minute heartbeats, this means paying for 48 full context windows per day per agent, with no server-side optimization possible. OpenRouter’s architecture as a routing layer fundamentally precludes session management – they would need to become an inference provider (not just a router) to offer this capability. This structural limitation makes OpenRouter the weakest performer on this dimension for agent workloads.

OpenAI does not offer session persistence on the standard `/v1/chat/completions` endpoint. However, the Assistants API provides thread-based state management where conversations persist server-side and subsequent messages are appended to existing threads. This is functionally similar to Gonka’s sessions but requires using a different API surface (Assistants API vs Chat Completions). For OpenClaw developers already using the chat completions endpoint, switching to Assistants API requires code changes and introduces a different request/response format. The TIE verdict reflects this: the capability exists but with integration friction.

Anthropic offers no session management. Each request is stateless. Anthropic’s prompt caching (covered under Memory/Context Management) partially compensates by reducing the cost of re-sent context, but the tokens are still transmitted and processed – caching reduces cost, not bandwidth. No equivalent to Gonka’s sessions or OpenAI’s Assistants threads exists.

Together AI offers no session management. Like Anthropic, each request is fully stateless. Agents must manage all conversation state client-side. Together AI’s focus on raw inference performance (low per-token cost, model variety) does not extend to agent-specific features.

Verdict rationale: Gonka’s server-side sessions directly address the single most expensive pattern in OpenClaw agent workloads (heartbeat context re-sends). No other compared provider offers an equivalent feature on the standard chat completions API surface. OpenAI’s Assistants API is the closest alternative but requires API surface migration.

2. Model Tiering / Auto-Routing

What OpenClaw agents need: A well-designed agent does not use the same model for every subtask. Classification (“Is this a code question or a general question?”) is a simple task that can be handled by a cheap, fast model. Complex

reasoning (“Debug this race condition in the distributed system”) requires a strong, expensive model. OpenClaw agents that use a single model for all tasks either overpay (using GPT-5.4 for classification) or underperform (using GPT-4o mini for complex reasoning). Automatic tiering – where the infrastructure routes requests to the appropriate model tier based on task complexity – eliminates this tradeoff without requiring the agent developer to build routing logic.

How each provider delivers:

Gonka implements 3-tier auto-routing through the X-Gonka-Tier header and automatic content pattern matching (built in v1.2, infrastructure/agent/tiering.py). An agent can send X-Gonka-Tier: lite for simple tasks (routed to a heavily quantized K2.5), X-Gonka-Tier: mid for moderate tasks, or X-Gonka-Tier: full for complex reasoning (routed to the full K2.5 model). Alternatively, the gateway can analyze request content patterns and auto-route without the header. This is a genuine agent-facing feature: the agent itself (not just the developer) makes runtime tier decisions programmatically. The limitation is that all three tiers are quantization variants of the same K2.5 model, so the capability range is narrower than routing between fundamentally different models (e.g., GPT-4o mini vs GPT-5.4).

OpenRouter offers manual model selection from a 500+ model catalog. A developer can configure their agent to use different models for different tasks, but this requires explicit model ID specification in each request. OpenRouter does not analyze request content or automatically route to an appropriate tier. The developer must build all routing logic client-side. OpenRouter’s “model routing” is a manual catalog lookup, not automatic tiering. For agent developers who want to set up tiering once and let the infrastructure handle it, OpenRouter requires custom middleware that Gonka provides natively. OpenRouter’s vast model catalog is its strength on the Model Breadth dimension, but on automatic tiering it offers no native capability. This is a structural gap: as a routing layer, OpenRouter’s value is in aggregation, not in making intelligent routing decisions on behalf of agents.

OpenAI requires manual model selection. An agent must explicitly specify `model: "gpt-5.4"` or `model: "gpt-4o-mini"` per request. No automatic routing exists. OpenAI’s model lineup does offer a natural tiering (4o-mini for cheap, 4o for balanced, 5.4 for strong), but the developer must implement all routing logic. No API mechanism exists for “send this request and let the infrastructure pick the right model.”

Anthropic similarly requires manual model selection (Haiku, Sonnet, Opus). No auto-routing capability.

Together AI requires manual model selection from its 200+ open model catalog. No tiering or auto-routing mechanism.

Verdict rationale: Gonka is the only compared provider that offers infrastructure-level automatic model tiering. While the current implementation is limited to K2.5 quantization variants (not fundamentally different models), the pattern – header-based tier selection + content-aware auto-routing – is a genuine differentiator that directly addresses agent developers’ need for cost-optimal model selection without custom routing code.

3. Tool Calling

What OpenClaw agents need: Tool calling (function calling) is the mechanism by which agents interact with external systems – reading files, running searches, executing code, calling APIs. An OpenClaw agent making 3-10x more LLM calls than a chatbot (source: LangChain State of Agent Engineering) uses tool calling extensively. The quality of tool calling matters: does the model reliably select the right tool, format parameters correctly, and handle multi-step tool chains without hallucinating function names or arguments? For agent workloads, tool calling reliability is arguably the single most important model capability.

How each provider delivers:

Gonka serves K2.5 which supports native tool calling with OpenAI-compatible function calling format. K2.5 demonstrates stability with 200-300 sequential tool calls in testing (v1.2 integration tests, infrastructure/tests/test_openclaw.py). This is strong performance – most agent workflows involve fewer than 50 sequential calls. The limitation is that tool calling quality is model-dependent (K2.5’s quality, not Gonka’s infrastructure), and K2.5 is a newer model with less real-world tool calling validation than GPT-4’s multi-year track record.

OpenRouter passes through whatever tool calling the underlying model provider supports. If the developer routes through OpenRouter to OpenAI’s GPT-5.4, they get OpenAI’s tool calling. If they route to an open model with weaker tool calling, they get that quality. OpenRouter adds no tool calling capability of its own. The variability is the issue: an agent using OpenRouter with automatic model selection cannot guarantee consistent tool calling quality across models. OpenRouter does document which models support function calling, which helps developers make informed selections, but the quality guarantee is only as strong as the chosen model.

OpenAI delivers best-in-class function calling with structured outputs (guaranteed JSON schema compliance). GPT-5.4 and GPT-4o reliably select tools, format parameters, and handle complex multi-tool workflows. Structured outputs eliminate parsing errors – the model’s response is guaranteed to match the specified JSON schema. This is the benchmark

against which all other tool calling implementations are measured. For agent developers who need tool calling to work reliably at scale, OpenAI sets the standard.

Anthropic offers strong tool use that has matured significantly through 2025-2026. Claude models handle multi-tool workflows well, and structured output features are in beta. Tool use quality is close to OpenAI's but slightly less mature in edge cases (complex nested schemas, very long tool chains). The TIE verdict reflects the practical reality that for most agent workflows, Anthropic's tool calling is reliable and productive.

Together AI supports tool calling for compatible models (Llama 4, K2.5, etc.) via OpenAI-compatible API. Quality depends on the model chosen – Together AI's infrastructure faithfully passes through the model's native capabilities without adding or degrading tool calling quality.

Verdict rationale: OpenAI wins because structured outputs with guaranteed schema compliance is a measurable, concrete advantage for agent workloads. Gonka (via K2.5), Anthropic, and Together AI all offer functional tool calling that serves most agent use cases, earning TIEs. OpenRouter's passthrough model creates variability that prevents a WIN but does not constitute a clear loss.

4. Streaming

What OpenClaw agents need: Server-Sent Events (SSE) streaming allows agents to begin processing partial responses before the full generation completes. For agent workloads, streaming reduces perceived latency and enables progressive processing (e.g., an agent can start parsing a tool call response while the model is still generating the reasoning text). All major inference providers support SSE streaming, and OpenClaw expects it via the standard OpenAI streaming format.

How each provider delivers:

All five compared providers support SSE streaming via the OpenAI-compatible `stream: true` parameter. Gonka implements streaming through vLLM's native SSE support. OpenRouter passes through streaming from upstream providers. OpenAI, Anthropic, and Together AI all provide native streaming implementations.

The practical differences are minor: latency to first token varies (Groq, not in this comparison, sets the benchmark at 0.13s TTFT), and some providers handle streaming edge cases (tool call streaming, multi-turn streaming) slightly differently. But for the standard agent use case – receiving tokens as they are generated – all five providers deliver equivalent functionality.

Verdict rationale: Streaming is table stakes. All providers deliver it. No meaningful differentiation exists on this dimension for agent workloads. Universal TIE.

5. Memory / Context Management

What OpenClaw agents need: Long-running agents accumulate knowledge over time – user preferences, project context, prior decisions, learned patterns. Without server-side memory management, this knowledge must be reconstructed from conversation history on every invocation, or stored and managed entirely client-side. Two mechanisms address this: prompt caching (reducing the cost of re-sending known context) and persistent memory APIs (storing and retrieving knowledge across sessions).

OpenClaw agents are particularly affected because heartbeats (every 30 minutes) resend the full system prompt and workspace context – approximately 9,600 tokens per heartbeat (source: OpenClaw pricing guides). Over a day with 48 heartbeats, that is 460,800 tokens of repeated context. Any mechanism that reduces the cost or eliminates the need for this repetition directly impacts agent economics.

How each provider delivers:

Gonka offers a `/v1/memory` API (built in v1.2) that provides persistent key-value memory storage accessible via API. Agents can store facts, preferences, and context across sessions. The current search implementation uses TF-IDF (v1.2 tech debt – noted in PROJECT.md), which provides functional keyword-based retrieval but lower recall than vector embedding approaches. For queries like “What did the user say about their deployment preferences?”, TF-IDF may miss semantically similar but lexically different stored memories. This is a genuine capability that no other compared provider offers in the same form, but the quality gap vs vector search limits its current practical advantage. Combined with Gonka's session persistence (covered in dimension 1), agents can maintain both short-term (session) and long-term (memory) context.

OpenRouter offers no memory management. As a stateless routing layer, OpenRouter has no mechanism for storing or retrieving agent context across requests. All memory management must be implemented client-side. For OpenClaw developers using OpenRouter, this means either building custom memory infrastructure or accepting the full cost of context re-transmission on every call. OpenRouter’s architecture as a thin proxy makes adding memory management a fundamental architectural change, not a feature addition. This is the weakest position among compared providers on this dimension alongside Together AI, as both offer zero server-side context management.

OpenAI provides native prompt caching that automatically identifies repeated prefixes in consecutive requests and caches them server-side. Cached tokens are billed at 50% of the standard input rate. For OpenClaw heartbeats that resend the same 9,600-token system prompt every 30 minutes, this means the second and subsequent heartbeats pay half price on the repeated portion. This is a significant economic benefit: at 48 heartbeats/day, prompt caching saves approximately 23 heartbeats’ worth of input token costs. The caching is automatic – no developer action required beyond sending requests with consistent prefixes. This addresses the cost problem (paying for repeated context) even though it does not address the architectural problem (agents still re-send the context).

Anthropic offers the most aggressive prompt caching economics: cached tokens are billed at 90% discount (10% of standard rate), with a 25% write premium on the first cache write. For OpenClaw agents with frequent heartbeats, this means the first heartbeat’s context costs 125% of standard, but all subsequent heartbeats for that context cost only 10%. Over 48 daily heartbeats, the effective savings are dramatic – approximately 89% reduction in input token costs for the heartbeat context portion. This makes Anthropic the most cost-effective provider for the specific pattern of repeated context re-sending, despite Anthropic’s premium per-token rates. The WIN verdict reflects that for the specific workload pattern of OpenClaw agents (frequent, large context re-sends), Anthropic’s caching economics are the strongest among compared providers.

Together AI offers no memory management or prompt caching. Each request is processed independently at full token cost. Like OpenRouter, all context management is client-side.

Verdict rationale: OpenAI and Anthropic win with fundamentally different approaches – OpenAI with automatic 50% caching and Anthropic with 90% caching at a write premium. Gonka’s memory API offers a unique capability (persistent long-term memory) but TF-IDF search quality limits its current advantage to a TIE. OpenRouter and Together AI lose by offering nothing on this dimension.

6. Pricing Model

What OpenClaw agents need: OpenClaw agent workloads have a distinctive cost profile that differs from chatbot usage. Agents make 3-10x more LLM calls per task than conversational chatbots (source: LangChain State of Agent Engineering). Heartbeats add a recurring fixed cost of approximately 9,600 input tokens every 30 minutes (460,800 tokens/day per agent). Multi-agent setups multiply these costs linearly. Community reports indicate monthly costs ranging from \$5-30/month for casual single-agent setups to \$200-1,000+/month for heavy multi-agent production deployments (source: .planning/research/STACK.md, OpenClaw community reports).

For agent developers, the pricing model matters as much as the per-token rate. Hidden costs (markup percentages, rate limiting throttling, minimum commitments), pricing transparency (can the developer estimate costs before committing?), and pricing predictability (does pricing change frequently?) all affect the total cost of operating agent infrastructure.

How each provider delivers:

Gonka targets per-token pricing with a potential cost advantage from decentralized compute (Gonka’s own estimate: 50-70% cheaper than centralized). However, Gonka’s pricing is not yet published. No per-token rates, no pricing page, no cost calculator exists. This makes it impossible to make verified cost comparisons. The TIE verdict reflects a structural reality: the pricing model (per-token, no middleman markup) is sound, but without published prices, developers cannot evaluate Gonka on cost. Any cost advantage claim remains contingent on final pricing decisions.

OpenRouter uses per-token passthrough pricing plus a 5.5% credit markup. When a developer purchases \$100 of OpenRouter credits, they receive \$94.50 in inference capacity. This markup is documented but not prominently displayed – a developer comparing OpenRouter’s listed model prices against direct provider prices will see identical per-token rates, only discovering the effective 5.5% premium when purchasing credits. For K2.5 specifically, OpenRouter routes to upstream providers (Together AI, DeepInfra, etc.) at approximately \$0.60/\$2.50 per 1M tokens, plus the 5.5% markup, making the effective rate roughly \$0.63/\$2.64. For agents making thousands of calls monthly, this markup compounds. However, OpenRouter’s pricing is transparent (published per-model rates), predictable, and requires no commitment. The TIE reflects the tradeoff: slightly higher effective cost due to markup, but excellent transparency and flexibility.

OpenAI offers straightforward per-token pricing with published rates for each model. Volume discounts are available for high-usage customers. Pricing is transparent, well-documented, and includes a usage dashboard with spending alerts.

GPT-4o at \$2.50/\$10.00 per 1M tokens and GPT-5.4 at higher rates represent premium pricing for proprietary models. For OpenCrew agents, the prompt caching discount (covered under Memory) effectively reduces costs by 50% on repeated context. The TIE verdict reflects the balance: premium per-token rates offset by transparent pricing, volume discounts, and caching benefits.

Anthropic charges the highest per-token rates among compared providers for its flagship model: Opus 4 at \$15.00/\$75.00 per 1M tokens. For agent workloads that make many calls, these rates create the highest raw cost. Prompt caching (90% discount on cached tokens) significantly reduces effective costs for repeat-context patterns, but the base rates remain the steepest. Sonnet 4 at lower rates provides a more cost-effective option, but agents needing Opus-level reasoning face premium pricing. The LOSE verdict reflects that for agent workloads specifically – which multiply per-token costs through high call volumes – Anthropic’s pricing creates the highest baseline cost, even accounting for caching benefits.

Together AI operates its own GPU clusters (H100/H200/B200) with no middleman markup. K2.5 is priced at \$0.50/\$2.50 per 1M tokens – the lowest verified price for K2.5 inference among compared providers (source: together.ai/pricing, Artificial Analysis). Llama 4 Maverick at \$0.27/\$0.85 per 1M tokens represents the floor for capable open models. Together AI’s pricing advantage comes from vertical integration (own hardware, no cloud provider margin, no routing markup). The WIN verdict reflects that Together AI offers the lowest verified per-token prices for the models most relevant to agent workloads, with transparent pricing and no hidden costs.

Verdict rationale: Together AI wins with the lowest verified pricing and no hidden markup. OpenAI and OpenRouter earn TIEs for different reasons (premium but transparent, and affordable but with markup). Anthropic loses on raw cost for high-volume agent workloads. Gonka’s TBD pricing prevents evaluation – the TIE is provisional, reflecting a sound pricing model structure without verifiable numbers.

7. Uptime / Reliability

What OpenCrew agents need: Agent reliability requirements are fundamentally different from chatbot reliability requirements. A chatbot user experiencing a timeout can retry. An agent in the middle of a multi-step workflow – having already read files, planned an approach, and completed 3 of 5 subtasks – loses all progress if the inference provider is unavailable for the 4th call. Agent workflows are stateful and sequential; a single failed call can cascade into a wasted pipeline of prior successful calls. OpenCrew agents running on heartbeats need near-continuous availability: a 10-minute outage means missed heartbeats and potentially stale agent state.

For production agent deployments, uptime SLAs and historical reliability track records are not nice-to-haves – they determine whether the provider can be trusted with mission-critical agent workflows.

How each provider delivers:

Gonka has no published SLA, no public status page, and no historical uptime data. The network is unproven at production scale. Decentralized infrastructure introduces inherent variability: heterogeneous GPU hardware, public internet routing between nodes, and variable node availability create reliability challenges that centralized providers do not face. v1.2 integration tests demonstrate functional correctness but not production reliability. The LOSE verdict is clear: an agent developer evaluating Gonka for production use has zero reliability data to base a decision on. This is Gonka’s most significant competitive disadvantage – not because the infrastructure is necessarily unreliable, but because the absence of evidence is itself evidence of risk.

OpenRouter has operated for multiple years with a generally reliable track record. However, OpenRouter does not publish a formal SLA. Free tier users experience throttling and degradation under load. As a routing layer, OpenRouter’s reliability depends on both its own infrastructure and the upstream providers it routes to – a double dependency. If OpenRouter is up but the upstream provider is down, the request still fails. OpenRouter mitigates this with automatic fallback routing (switching to alternative providers for the same model), which is a genuine reliability advantage over direct provider access. The TIE verdict reflects the balance: operational maturity without formal guarantees.

OpenAI sets the industry standard for inference reliability. A 99.9%+ SLA backed by global, multi-region infrastructure with automatic failover. OpenAI’s status page (status.openai.com) provides real-time and historical uptime data. For agent developers, OpenAI’s reliability means confidence that step 4 of a 5-step agent workflow will not fail due to infrastructure unavailability. The WIN verdict is straightforward: no other compared provider offers comparable reliability guarantees with comparable track record.

Anthropic delivers production-grade reliability with enterprise SLAs and consistent infrastructure. While Anthropic has experienced occasional outages, its overall track record is strong, and enterprise customers receive formal uptime commitments. The WIN verdict reflects that Anthropic meets the reliability bar for production agent deployments, matching OpenAI on this dimension.

Together AI operates its own GPU clusters, providing infrastructure control that routing layers lack. While no formal SLA is published, Together AI's track record on dedicated infrastructure is generally reliable. The TIE verdict reflects solid operational performance without formal guarantees.

Verdict rationale: OpenAI and Anthropic win with formal SLAs and proven track records. OpenRouter and Together AI earn TIEs for reliable operation without formal guarantees. Gonka loses because no reliability data exists, and decentralized infrastructure introduces variability risks that have not been publicly measured or mitigated.

8. Model Breadth

What OpenCrew agents need: Different agent tasks benefit from different models. Code generation may work best on Kimi K2.5 or Claude. Quick classification may be cheapest on Llama 4 Maverick. Complex reasoning may require GPT-5.4 or Opus 4. A provider offering multiple models allows agent developers to route different subtasks to the optimal model without managing multiple provider integrations. Model breadth also provides fallback resilience: if the primary model is degraded, the agent can automatically switch to an alternative.

For OpenCrew agents specifically, model breadth enables the multi-model tiering pattern where cheap models handle simple tasks and expensive models handle complex ones – the cost optimization strategy that agent developers consistently cite as a priority.

How each provider delivers:

Gonka serves a single model: Kimi K2.5, available in 3 quantization tiers (lite, mid, full). While K2.5 is a strong agent model (76.8% SWE-Bench, native Agent Swarm, 131K context window), the single-model limitation creates several problems for agent developers. First, no fallback: if K2.5 is degraded, there is no alternative model to route to. Second, limited task specialization: all tasks – from simple classification to complex reasoning – use variants of the same model, limiting the cost spread between tiers. Third, model risk: if K2.5 is superseded by a clearly superior model, Gonka has no alternative to offer until a new model is deployed. The LOSE verdict reflects a genuine competitive disadvantage that matters for production agent systems.

OpenRouter offers 500+ models from all major providers (OpenAI, Anthropic, Google, Meta, Mistral, and dozens of open-source models). A single OpenRouter API key provides access to every model, with consistent API formatting across providers. For agent developers, this means maximum flexibility: route classification to a cheap model, reasoning to GPT-5.4, and code generation to K2.5 – all through one API. OpenRouter's model breadth is its primary competitive advantage and the reason it is built into OpenCrew as a default provider. The WIN verdict is unambiguous: no other compared provider offers comparable model selection. OpenRouter's breadth is particularly threatening to Gonka because it provides the same K2.5 access (via upstream providers like Together AI and DeepInfra) plus 499 other models – making Gonka's single-model position redundant for developers who already have OpenRouter configured.

OpenAI offers approximately 10 proprietary models spanning different capability tiers (GPT-5.4, GPT-4o, GPT-4o mini, embeddings, DALL-E, etc.). While the catalog is small numerically, each model serves a distinct purpose and quality tier. For agent developers, OpenAI's lineup provides genuine tiering capability (mini for cheap tasks, 4o for balanced, 5.4 for strong). The TIE reflects that while the catalog is small, it is well-differentiated and covers the primary agent use cases.

Anthropic offers approximately 5 models (Opus 4, Sonnet 4, Haiku 3.5, plus older versions). The smallest catalog among compared providers. However, like OpenAI, the models are well-differentiated across capability tiers (Haiku for fast/cheap, Sonnet for balanced, Opus for reasoning-heavy). The LOSE verdict reflects the limited options, though the quality of each model partially compensates.

Together AI offers 200+ open-source models including K2.5, Llama 4 variants, and fine-tuning options. While not as broad as OpenRouter (which aggregates providers including Together AI), the selection is substantial and covers most agent use cases. The TIE verdict reflects strong model variety without matching OpenRouter's exhaustive catalog.

Verdict rationale: OpenRouter wins decisively with 500+ models. Together AI and OpenAI earn TIEs for strong but smaller catalogs. Anthropic and Gonka lose – Anthropic for a small but high-quality lineup, Gonka for the more limiting single-model position with no alternatives.

Key Takeaways

1. **OpenClaw agent developers need session persistence and automatic model tiering more than any other agent-specific feature, and Gonka is the only provider that delivers both natively.** No compared provider offers server-side session management on the standard chat completions API or automatic tier routing based on task complexity. These capabilities directly address the two most expensive patterns in agent workloads: heartbeat context re-sending and using expensive models for simple tasks.
2. **Gonka's biggest competitive liabilities are not feature gaps but trust gaps.** Model breadth (1 vs 500+) and reliability (no SLA vs 99.9%+) are not features Gonka can easily add – they require fundamentally different infrastructure (multi-model serving) and time (uptime track record). Agent developers will not adopt a provider they cannot trust with production workloads, regardless of how innovative its agent features are. The path forward is to establish reliability credibility first (published uptime stats, transparent status page), then expand model breadth strategically.
3. **OpenRouter is Gonka's primary competitive threat because it occupies the “default OpenClaw provider” position.** OpenRouter is built into OpenClaw, requiring zero configuration. Gonka requires manual JSON provider configuration. For an OpenClaw developer choosing an inference provider, the path of least resistance is OpenRouter. Gonka must either become a built-in OpenClaw provider (eliminating the configuration friction) or demonstrate agent-specific advantages compelling enough to justify the manual setup.
4. **Prompt caching (OpenAI, Anthropic) is a partial substitute for Gonka's sessions, weakening the differentiation.** OpenAI's 50% caching discount and Anthropic's 90% caching discount reduce the cost of heartbeat context re-sends without requiring server-side session management. While Gonka's sessions provide a cleaner architectural solution (the agent does not re-send context at all), the economic benefit of caching narrows Gonka's cost advantage on this dimension. Gonka's session value proposition must emphasize the architectural benefit (simpler agent code, reduced bandwidth) not just cost savings.
5. **Gonka cannot yet compete on pricing claims because its pricing is not published.** Every cost comparison in this analysis uses “TBD” or “contingent on final pricing” for Gonka. Until per-token rates are set and publicly visible, the “50-70% cheaper” positioning is unverifiable and should not be used in any external-facing materials. Competitors like Together AI (\$0.50/\$2.50 for K2.5) set the pricing benchmark that Gonka must beat to justify its cost advantage narrative.

Source Citations

Source	Used For	Confidence
.planning/research/STACK.md	OpenClaw provider architecture, competitive pricing data, community cost reports	HIGH
.planning/research/FEATURES.md	Feature landscape, decision criteria, gap analysis	HIGH
.planning/research/ARCHITECTURE.md	GTM framework, competitive analysis structure, persona definitions	HIGH
.planning/research/PITFALLS.md	Anti-patterns, trust barriers, recovery strategies	HIGH
infrastructure/agent/sessions.py	Gonka session persistence implementation details	HIGH (verified against codebase)
infrastructure/agent/tiering.py	Gonka auto-routing implementation details	HIGH (verified against codebase)
infrastructure/agent/memory.py	Gonka memory API implementation details	HIGH (verified against codebase)
infrastructure/tests/test_openclaw.py	Gonka OpenClaw integration test results	HIGH (verified against codebase)
OpenRouter pricing page (openrouter.ai/pricing)	5.5% credit markup, per-token pass-through rates	MEDIUM (accessed April 2026, subject to change)
Together AI pricing page (together.ai/pricing)	K2.5 at \$0.50/\$2.50, Llama 4 Maverick at \$0.27/\$0.85	MEDIUM (accessed April 2026)
OpenAI API pricing (platform.openai.com/pricing)	GPT-4o at \$2.50/\$10.00, prompt caching at 50% discount	MEDIUM (accessed April 2026)
Anthropic API pricing (anthropic.com/pricing)	Opus 4 at \$15.00/\$75.00, prompt caching at 90% discount / 25% write premium	MEDIUM (accessed April 2026)
Artificial Analysis (artificialanalysis.ai)	Multi-provider K2.5 pricing comparison	MEDIUM
OpenClaw pricing guides (multiple community sources)	9,600 tokens/turn overhead, heartbeat costs, monthly ranges	MEDIUM-HIGH
LangChain State of Agent Engineering	3-10x more LLM calls for agents vs chatbots	MEDIUM